

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №4 по курсу
«Операционные системы»

Группа: М8О-213Б-23

Студент: Барменков А.С.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 23.12.24

Москва, 2024

Постановка задачи

Вариант 1.

Требуется создать две динамические библиотеки, реализующие два аллокатора: списки свободных блоков (первое подходящее) и блоки по 2^n .

Общий метод и алгоритм решения

Использованные системные вызовы:

1. ***int munmap(void addr, size_t length);** - Удаляет отображения, созданные с помощью mmap.
2. ***int dlclose(void handle);** - Закрывает динамическую библиотеку, открытую с помощью dlopen, и освобождает ресурсы, связанные с этим дескриптором.
3. ****void dlopen(const char filename, int flag);** - Открывает динамическую библиотеку и возвращает дескриптор для последующего использования.
4. ****void mmap(void addr, size_t length, int prot, int flags, int fd, off_t offset);** – создает новое отображение памяти или изменяет существующее.
5. **int write(int _Filehandle, const void *_Buf, unsigned int _MaxCharCount)** – выводит информацию в Filehandle.

Описание программы

1. main.c

Открывает динамические библиотеки и получает нужные функции. Если в библиотеке не нашлось нужных функций, то вместо них будут использоваться аварийные оберточные функции. Далее как пример функция выделяет и освобождает память массива.

2. degree2.c

Файл в котором реализована логика работы аллокатора блоками по 2^n .

- 1) Вся память при инициализации разбивается на блоки которые равны степени двойки.
- 1) Все блоки хранятся в списке свободных элементов.
- 2) Каждый блок хранит указатель на следующий свободный блок.
- 3) При освобождении нужно добавить этот блок в список свободных элементов в нужную позицию.
- 4) Для выделения памяти выбираем блок $N[\log_2(\text{size})]$ и возвращаем указатель на первый элемент, помечая блок занятым.

3. List_allocator.c

Файл в котором реализована логика работы аллокатора на списках свободных блоков (наиболее подходящее).

- 1) Все свободные блоки организованы в список
- 2) В блоке хранится его размер и указатель на следующий свободный блок
- 3) Для выделения памяти проходимся по всему списку свободных блоков и выбираем минимальный блок, который больше или равен по размеру нужного блока. Помечаем блок, как занятый и убираем из списка.
- 4) При освобождении памяти возвращаем блок в список свободных элементов. И при возможности сливаем рядом стоящие блоки.

Код программы

main.c

```
#include <dlfcn.h>
#include <math.h>
#include <stddef.h>
#include <stdint.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <unistd.h>

typedef struct Allocator {
    void (*allocator_create)(void *addr, size_t size);
    void (*allocator_alloc)(void *allocator, size_t size);
    void (*allocator_free)(void *allocator, void *ptr);
    void (*allocator_destroy)(void *allocator);
} Allocator;

void *standard_allocator_create(void *memory, size_t size) {
    (void)size;
    (void)memory;
    return memory;
}

void *standard_allocator_alloc(void *allocator, size_t size) {
    (void)allocator;
    uint32_t *memory =
        mmap(NULL, size + sizeof(uint32_t), PROT_READ | PROT_WRITE,
            MAP_SHARED | MAP_ANONYMOUS, -1, 0);
    if (memory == MAP_FAILED) {
        return NULL;
    }
    *memory = (uint32_t)(size + sizeof(uint32_t));
    return memory + 1;
}

void standard_allocator_free(void *allocator, void *memory) {
    (void)allocator;
    if (memory == NULL) return;
    uint32_t *mem = (uint32_t *)memory - 1;
    munmap(mem, *mem);
}

void standard_allocator_destroy(void *allocator) { (void)allocator; }

void load_allocator(const char *library_path, Allocator *allocator) {
    void *library = dlopen(library_path, RTLD_LOCAL | RTLD_NOW);
    if (library_path == NULL || library_path[0] == '\0' || !library) {
        char message[] = "WARNING: failed to load shared library\n";
        write(STDERR_FILENO, message, sizeof(message) - 1);
        allocator->allocator_create = standard_allocator_create;
        allocator->allocator_alloc = standard_allocator_alloc;
    }
}
```

```

        allocator->allocator_free = standard_allocator_free;
        allocator->allocator_destroy = standard_allocator_destroy;
        return;
    }

    allocator->allocator_create = dlsym(library, "allocator_create");
    allocator->allocator_alloc = dlsym(library, "allocator_alloc");
    allocator->allocator_free = dlsym(library, "allocator_free");
    allocator->allocator_destroy = dlsym(library, "allocator_destroy");

    if (!allocator->allocator_create || !allocator->allocator_alloc ||
        !allocator->allocator_free || !allocator->allocator_destroy) {
        const char msg[] = "Error: failed to load all allocator functions\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        dlclose(library);
        return;
    }
}

void write_message(const char *message) {
    write(STDOUT_FILENO, message, strlen(message));
}

void write_address(const char *prefix, int index, void *address) {
    char buffer[64];
    int len = 0;
    while (prefix[len] != '\0') {
        buffer[len] = prefix[len];
        len++;
    }
    buffer[len++] = ' ';
    if (index < 10) {
        buffer[len++] = '0' + index;
    } else {
        buffer[len++] = '0' + (index / 10);
        buffer[len++] = '0' + (index % 10);
    }
    buffer[len++] = ' ';
    buffer[len++] = 'a';
    buffer[len++] = 'd';
    buffer[len++] = 'd';
    buffer[len++] = 'r';
    buffer[len++] = 'e';
    buffer[len++] = 's';
    buffer[len++] = 's';
    buffer[len++] = ':';
    buffer[len++] = ' ';
    uintptr_t addr = (uintptr_t)address;
    for (int i = (sizeof(uintptr_t) * 2) - 1; i >= 0; --i) {
        int nibble = (addr >> (i * 4)) & 0xF;
        buffer[len++] = (nibble < 10) ? ('0' + nibble) : ('a' + (nibble - 10));
    }
    buffer[len++] = '\n';
    write(STDOUT_FILENO, buffer, len);
}

```

```

}

int main(int argc, char **argv) {
    const char *library_path = (argc > 1) ? argv[1] : NULL;
    Allocator allocator_api;
    load_allocator(library_path, &allocator_api);

    size_t size = 4096;
    void *addr = mmap(NULL, size, PROT_READ | PROT_WRITE,
                      MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    if (addr == MAP_FAILED) {
        char message[] = "mmap failed\n";
        write(STDERR_FILENO, message, sizeof(message) - 1);
        return EXIT_FAILURE;
    }

    void *allocator = allocator_api.allocator_create(addr, size);
    if (!allocator) {
        char message[] = "Failed to initialize allocator\n";
        write(STDERR_FILENO, message, sizeof(message) - 1);
        munmap(addr, size);
        return EXIT_FAILURE;
    }

    void *blocks[10];
    size_t block_sizes[12] = {12, 13, 13, 24, 40, 56, 100, 120, 400, 120, 120, 120};

    int alloc_failed = 0;
    for (int i = 0; i < 12; ++i) {
        blocks[i] = allocator_api.allocator_alloc(allocator, block_sizes[i]);
        if (blocks[i] == NULL) {
            alloc_failed = 1;
            char alloc_fail_message[] = "Memory allocation failed\n";
            write(STDERR_FILENO, alloc_fail_message,
                  sizeof(alloc_fail_message) - 1);
            break;
        }
    }

    if (!alloc_failed) {
        char alloc_success_message[] = "Memory allocated successfully\n";
        write(STDOUT_FILENO, alloc_success_message,
              sizeof(alloc_success_message) - 1);

        for (int i = 0; i < 12; ++i) {
            write_address("Block", i + 1, blocks[i]);
        }
    }

    for (int i = 0; i < 12; ++i) {
        if (blocks[i] != NULL)
            allocator_api.allocator_free(allocator, blocks[i]);
    }
}

```

```

    char free_message[] = "Memory freed\n";
    write(STDOUT_FILENO, free_message, sizeof(free_message) - 1);

    allocator_api.allocator_destroy(allocator);

    char exit_message[] = "Program exited successfully\n";
    write(STDOUT_FILENO, exit_message, sizeof(exit_message) - 1);

    return EXIT_SUCCESS;
}

```

list_allocator.c

```

#include <stddef.h>

typedef struct Allocator {
    void* memory_start;
    size_t memory_size;
    void* free_list;
} Allocator;

typedef struct FreeBlock {
    size_t size;
    struct FreeBlock* next;
} FreeBlock;

Allocator* allocator_create(void* const memory, const size_t size) {
    if (!memory || size < sizeof(FreeBlock)) {
        return NULL;
    }

    Allocator* allocator = (Allocator*)memory;
    allocator->memory_start = (char*)memory + sizeof(Allocator);
    allocator->memory_size = size - sizeof(Allocator);
    allocator->free_list = allocator->memory_start;

    FreeBlock* initial_block = (FreeBlock*)allocator->memory_start;
    initial_block->size = allocator->memory_size;
    initial_block->next = NULL;

    return allocator;
}

void allocator_destroy(Allocator* const allocator) {
    allocator->memory_start = NULL;
    allocator->memory_size = 0;
    allocator->free_list = NULL;
}

void* allocator_alloc(Allocator* const allocator, const size_t size) {
    if (!allocator || size == 0) {
        return NULL;
    }
}

```

```

FreeBlock* prev = NULL;
FreeBlock* curr = (FreeBlock*)allocator->free_list;

while (curr) {
    if (curr->size >= size + sizeof(FreeBlock)) {
        if (curr->size > size + sizeof(FreeBlock)) {
            FreeBlock* new_block = (FreeBlock*)((char*)curr + sizeof(FreeBlock) +
size);

            new_block->size = curr->size - size - sizeof(FreeBlock);
            new_block->next = curr->next;

            curr->size = size;
            curr->next = new_block;
        }

        if (prev) {
            prev->next = curr->next;
        } else {
            allocator->free_list = curr->next;
        }

        return (char*)curr + sizeof(FreeBlock);
    }

    prev = curr;
    curr = curr->next;
}

return NULL;
}

void allocator_free(Allocator* const allocator, void* const memory) {
    if (!allocator || !memory) {
        return;
    }

    FreeBlock* block = (FreeBlock*)((char*)memory - sizeof(FreeBlock));
    block->next = (FreeBlock*)allocator->free_list;
    allocator->free_list = block;
}

```

degree2.c

```
#include <math.h>

#include <stdbool.h>

#include <stdint.h>

#include <stdlib.h>

#include <string.h>

#include <sys/mman.h>

#include <unistd.h>
```

```
#define MIN_BLOCK_SIZE 16 // Минимальный размер блока
```

```
int log2s(int n) {
    if (n == 0) {
        return -1;
    }

    int result = 0;
    while (n > 1) {
        n >>= 1;
        result++;
    }

    return result;
}
```

```
typedef struct BlockHeader {
    struct BlockHeader *next;
} BlockHeader;
```

```
// Структура аллокатора
```

```
typedef struct Allocator {
    BlockHeader **free_lists;
    size_t num_lists;
    void *base_addr;
    size_t total_size;
```

```
} Allocator;
```

```
Allocator *allocator_create(void *memory, size_t size) {  
    if (!memory || size < sizeof(Allocator)) {  
        return NULL;  
    }  
    Allocator *allocator = (Allocator *)memory;  
    allocator->base_addr = memory;  
    allocator->total_size = size;  
  
    size_t min_usable_size = sizeof(BlockHeader) + MIN_BLOCK_SIZE;  
  
    size_t max_block_size = (size < 32) ? 32 : size;  
  
    allocator->num_lists = (size_t)floor(log2s(max_block_size) / 2) + 3;  
    allocator->free_lists =  
        (BlockHeader **)((char *)memory + sizeof(Allocator));  
  
    for (size_t i = 0; i < allocator->num_lists; i++) {  
        allocator->free_lists[i] = NULL;  
    }  
  
    void *current_block = (char *)memory + sizeof(Allocator) +  
        allocator->num_lists * sizeof(BlockHeader *);  
    size_t remaining_size =  
        size - sizeof(Allocator) - allocator->num_lists * sizeof(BlockHeader *);  
  
    size_t block_size = MIN_BLOCK_SIZE;  
    while (remaining_size >= min_usable_size) {  
        if (block_size > remaining_size) {  
            break;  
        }  
    }
```

```

    if (block_size > max_block_size) {
        break;
    }

    if (remaining_size >= (block_size + sizeof(BlockHeader)) * 2) {
        for (int i = 0; i < 2; i++) {
            BlockHeader *header = (BlockHeader *)current_block;

            size_t index = (size == 0) ? 0 : (size_t)log2s(block_size);
            header->next = allocator->free_lists[index];
            allocator->free_lists[index] = header;

            current_block = (char *)current_block + block_size;
            remaining_size -= block_size;
        }
    } else {
        BlockHeader *header = (BlockHeader *)current_block;

        size_t index = (size == 0) ? 0 : (size_t)log2s(block_size);
        header->next = allocator->free_lists[index];
        allocator->free_lists[index] = header;

        current_block = (char *)current_block + remaining_size;
        remaining_size = 0;
    }

    block_size <= 1;
}

return allocator;
}

```

// Функция выделения памяти

```

void *allocator_alloc(Allocator *allocator, size_t size) {

```

```

if (!allocator || size == 0) {
    return NULL;
}

size_t index = (size == 0) ? 0 : log2s(size) + 1;
if (index >= allocator->num_lists) {
    index = allocator->num_lists;
}

bool flag = false;
if (allocator->free_lists[index] == NULL) {
    while (index <= allocator->num_lists) {
        if (allocator->free_lists[index] != NULL) {
            flag = true;
            break;
        } else {
            ++index;
        }
    }
    if (!flag) return NULL;
}

BlockHeader *block = allocator->free_lists[index];
allocator->free_lists[index] = block->next;

return (void *)((char *)block + sizeof(BlockHeader));
}

// Функция освобождения памяти
void allocator_free(Allocator *allocator, void *ptr) {
    if (!allocator || !ptr) {
        return;
    }

```

```

BlockHeader *block = (BlockHeader *)((char *)ptr - sizeof(BlockHeader));

size_t temp_size =
    (char *)block + sizeof(BlockHeader) - (char *)allocator->base_addr;
size_t temp = 32;

while (temp <= temp_size) {
    size_t next_size = temp << 1;
    if (next_size > temp_size) {
        break;
    }
    temp = next_size;
}

size_t index = (temp_size == 0) ? 0 : (size_t)log2s(temp);
if (index >= allocator->num_lists) {
    index = allocator->num_lists - 1;
}

block->next = allocator->free_lists[index];
allocator->free_lists[index] = block;
}

// Функция уничтожения аллокатора
void allocator_destroy(Allocator *allocator) {
    if (allocator) {
        munmap(allocator->base_addr, allocator->total_size);
    }
}

```

Протокол работы программы

```
artemdelgray@artemdelgray-VirtualBox:~/Загрузки/Telegram Desktop$ ./main ./degree2.so
Memory allocated successfully
Block 1 address: 00007db02788b080
Block 2 address: 00007db02788b070
Block 3 address: 00007db02788b650
Block 4 address: 00007db02788b0b0
Block 5 address: 00007db02788b110
Block 6 address: 00007db02788b0d0
Block 7 address: 00007db02788b1d0
Block 8 address: 00007db02788b150
Block 9 address: 00007db02788b650
Block 10 address: 00007db02788b350
Block 11 address: 00007db02788b250
Block 12 address: 00007db02788b450
Memory freed
Program exited successfully
artemdelgray@artemdelgray-VirtualBox:~/Загрузки/Telegram Desktop$ ./main ./list.so
Memory allocated successfully
Block 1 address: 00007c61aafab028
Block 2 address: 00007c61aafab044
Block 3 address: 00007c61aafab061
Block 4 address: 00007c61aafab07e
Block 5 address: 00007c61aafab0a6
Block 6 address: 00007c61aafab0de
Block 7 address: 00007c61aafab126
Block 8 address: 00007c61aafab19a
Block 9 address: 00007c61aafab222
Block 10 address: 00007c61aafab3c2
Block 11 address: 00007c61aafab44a
Block 12 address: 00007c61aafab4d2
Memory freed
Program exited successfully
```

Strace:

```
strace -f ./main ./list.so
```

```
execve("./main", ["/main", "/list.so"], 0x7ffd367236a0 /* 46 vars */) = 0
```

```
brk(NULL) = 0x645a839fc000
```

```
arch_prctl(0x3001 /* ARCH_??? */, 0x7ffd8b3dea70) = -1 EINVAL (Недопустимый аргумент)
```

```
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7176bc13d000
```

```
access("/etc/ld.so.preload", R_OK) = -1 ENOENT (Нет такого файла или каталога)
```

```
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
```

```
newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=58047, ...}, AT_EMPTY_PATH) = 0
```

```

mmap(NULL, 58047, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7176bc12e000
close(3) = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\3\0>\0\1\0\0\0P\237\2\0\0\0\0"..., 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) = 784
pread64(3, "\4\0\0\0 \0\0\0\5\0\0\0GNU\0\2\0\0\300\4\0\0\0\3\0\0\0\0\0\0"..., 48, 848) = 48
pread64(3, "\4\0\0\0\24\0\0\0\3\0\0\0GNU\0I\17\357\204\3$\f\221\2039x\324\224\323\236S"..., 68, 896) = 68
newfstatat(3, "", {st_mode=S_IFREG|0755, st_size=2220400, ...}, AT_EMPTY_PATH) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) = 784
mmap(NULL, 2264656, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7176bbe00000
mprotect(0x7176bbe28000, 2023424, PROT_NONE) = 0
mmap(0x7176bbe28000, 1658880, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7176bbe28000
mmap(0x7176bbbfd000, 360448, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1bd000) = 0x7176bbbfd000
mmap(0x7176bc016000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x215000) = 0x7176bc016000
mmap(0x7176bc01c000, 52816, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7176bc01c000
close(3) = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7176bc12b000
arch_prctl(ARCH_SET_FS, 0x7176bc12b740) = 0
set_tid_address(0x7176bc12ba10) = 3961
set_robust_list(0x7176bc12ba20, 24) = 0
rseq(0x7176bc12c0e0, 0x20, 0, 0x53053053) = 0
mprotect(0x7176bc016000, 16384, PROT_READ) = 0
mprotect(0x645a8380b000, 4096, PROT_READ) = 0
mprotect(0x7176bc177000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0

```

```

munmap(0x7176bc12e000, 58047) = 0

getrandom("\xba\x64\x35\x2c\xa2\x99\x99\xdc", 8, GRND_NONBLOCK) = 8

brk(NULL) = 0x645a839fc000

brk(0x645a83a1d000) = 0x645a83a1d000

openat(AT_FDCWD, "./list.so", O_RDONLY|O_CLOEXEC) = 3

read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0"..., 832) = 832

newfstatat(3, "", {st_mode=S_IFREG|0775, st_size=15272, ...}, AT_EMPTY_PATH) = 0

getcwd("/home/artemdelgray/\320\227\320\260\320\263\321\200\321\203\320\267\320\272\320\270
/Telegram Desktop", 128) = 53

mmap(NULL, 16424, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) =
0x7176bc138000

mmap(0x7176bc139000, 4096, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1000) = 0x7176bc139000

mmap(0x7176bc13a000, 4096, PROT_READ,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x2000) = 0x7176bc13a000

mmap(0x7176bc13b000, 8192, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x2000) = 0x7176bc13b000

close(3) = 0

mprotect(0x7176bc13b000, 4096, PROT_READ) = 0

mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0)
= 0x7176bc176000

write(1, "Memory allocated successfully\n", 30Memory allocated successfully
) = 30

write(1, "Block 1 address: 00007176bc17602"..., 34Block 1 address: 00007176bc176028
) = 34

write(1, "Block 2 address: 00007176bc17604"..., 34Block 2 address: 00007176bc176044
) = 34

write(1, "Block 3 address: 00007176bc17606"..., 34Block 3 address: 00007176bc176061
) = 34

write(1, "Block 4 address: 00007176bc17607"..., 34Block 4 address: 00007176bc17607e
) = 34

write(1, "Block 5 address: 00007176bc1760a"..., 34Block 5 address: 00007176bc1760a6

```

```

) = 34
write(1, "Block 6 address: 00007176bc1760d"..., 34Block 6 address: 00007176bc1760de
) = 34
write(1, "Block 7 address: 00007176bc17612"..., 34Block 7 address: 00007176bc176126
) = 34
write(1, "Block 8 address: 00007176bc17619"..., 34Block 8 address: 00007176bc17619a
) = 34
write(1, "Block 9 address: 00007176bc17622"..., 34Block 9 address: 00007176bc176222
) = 34
write(1, "Block 10 address: 00007176bc1763"..., 35Block 10 address: 00007176bc1763c2
) = 35
write(1, "Block 11 address: 00007176bc1764"..., 35Block 11 address: 00007176bc17644a
) = 35
write(1, "Block 12 address: 00007176bc1764"..., 35Block 12 address: 00007176bc1764d2
) = 35
write(1, "Memory freed\n", 13Memory freed
)
    = 13
write(1, "Program exited successfully\n", 28Program exited successfully
) = 28
exit_group(0)
    = ?
+++ exited with 0 +++

```

Вывод

В рамках лабораторной работы была разработана программа, демонстрирующая работу аллокатора передаваемого в качестве аргумента при вызове программы. Было реализовано 2 аллокатора и проведена работа по сравнении их работоспособности.